

```

# Step 5: Pad these numbers, then add them for the return value:
step1Padding = (len(x) - halfOfDigits) + (len(x) - halfOfDigits)
step1PaddedNum = int(padZeros(str(step1Result), step1Padding, 'right'))

step4Padding = (len(x) - halfOfDigits)
step4PaddedNum = int(padZeros(str(step4Result), step4Padding, 'right'))

print('Solved', x, 'x', y, '=', step1PaddedNum + step2Result + step4PaddedNum)

return step1PaddedNum + step2Result + step4PaddedNum ❸

# Example: 1357 x 2468 = 3349076
print('1357 * 2468 =', karatsuba(1357, 2468))

```

The JavaScript equivalent is in *karatsubaMultiplication.html*:

```

<script type="text/javascript">

// Create a lookup table of all single-digit multiplication products:
let MULT_TABLE = {}; ❶
for (let i = 0; i < 10; i++) {
  for (let j = 0; j < 10; j++) {
    MULT_TABLE[[i, j]] = i * j;
  }
}

function padZeros(numberString, numZeros, insertSide) {
  // Return a string padded with zeros on the left or right side.
  if (insertSide === "left") {
    return "0".repeat(numZeros) + numberString;
  } else if (insertSide === "right") {
    return numberString + "0".repeat(numZeros);
  }
}

function karatsuba(x, y) {
  // Multiply two integers with the Karatsuba algorithm. Note that
  // the * operator isn't used anywhere in this function.
  console.assert(Number.isInteger(x), "x must be an integer");
  console.assert(Number.isInteger(y), "y must be an integer");
  x = x.toString();
  y = y.toString();

```